

ROS 2 LAB #2: MOTION CONTROL OF TURTLE

Abdelbacet Mhamdi

Senior-lecturer, Dept. of EE

ISSET Bizerte — Tunisia

 a-mhamdi

Abstract — This lab establishes the foundation for building complex ROS 2 applications through custom node development and inter-node communication patterns. You will create your own ROS 2 package, write custom nodes, and implement publisher-subscriber communication patterns.

I. PROJECT OVERVIEW

A. Workspace Setup

Create a ROS 2 workspace directory structure:

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
```

The src directory will contain all your ROS 2 packages. This workspace structure is the standard convention for organizing ROS 2 projects.

B. Package Creation

Create a new Python package called turtle_controller:

```
ros2 pkg create --build-type ament_python
turtle_controller --dependencies rclpy
geometry_msgs turtlesim
```

--build-type ament_python Specifies a Python package

turtle_controller Package name

--dependencies Lists required packages

This creates the following structure:

```
turtle_controller/
├── package.xml
├── setup.py
├── setup.cfg
├── turtle_controller/
│   └── __init__.py
└── resource/
    └── turtle_controller
```

II. WRITING YOUR FIRST PUBLISHER NODE

A. Simple Velocity Publisher

Navigate to your package directory and create a publisher node:

```
cd ~/ros2_ws/src/turtle_controller/
turtle_controller
```

Create velocity_publisher.py:

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
import math

class VelocityPublisher(Node):
    def __init__(self):
        super().__init__('velocity_publisher')

        # Create publisher
        self.publisher_ = self.create_publisher(
            Twist,
            '/turtle1/cmd_vel',
            10
        )

        # Timer for periodic publishing
        timer_period = 0.1 # 10 Hz
        self.timer =
self.create_timer(timer_period,
self.timer_callback)

        # Initialize variables
        self.counter = 0.0

        self.get_logger().info('Velocity
Publisher Node Started')

    def timer_callback(self):
        msg= Twist()

        # Create circular motion
        msg.linear.x = 2.0
        msg.angular.z = 1.0 *
math.sin(self.counter)

        self.publisher_.publish(msg)
        self.counter += 0.1

        self.get_logger().info(
```

```

        f'Publishing:
linear.x={msg.linear.x:.2f},
angular.z={msg.angular.z:.2f}')

def main(args=None):
    rclpy.init(args=args)

    velocity_publisher = VelocityPublisher()

    try:
        rclpy.spin(velocity_publisher)
    except KeyboardInterrupt:
        pass
    finally:
        velocity_publisher.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

Node() class inheritance for ROS 2 functionality
create_publisher() method with message type, topic, and queue size
timer_callback() Timer-based callbacks for periodic execution
get_logger().info() Logging

B. Configuring the Package

Edit `setup.py` to include your executable:

```

entry_points={
    'console_scripts': [
        'velocity_publisher =
turtle_controller.velocity_publisher:main',
    ]
}

```

Task 1: Position Monitor Subscriber

Create `position_subscriber.py` file where you write the code for the position subscriber node. Update `setup.py` to include the new executable.

Task 2: Intelligent Turtle Controller

You are asked to develop an intelligent controller that both subscribes to position and publishes velocity commands. Call the file `smart_controller.py`.

III. BUILDING AND RUNNING YOUR PACKAGE

A. Build Process

Build your package from the workspace root:

```

cd ~/ros2_ws
colcon build --packages-select turtle_controller

```

Source the workspace:

```
source ~/ros2_ws/install/setup.bash
```

i Info

Open a new terminal for each node and source the workspace again.

B. Running Individual Nodes

Start TurtleSim:

```
ros2 run turtlesim turtlesim_node
```

Run your velocity publisher:

```
ros2 run turtle_controller velocity_publisher
```

Run your position subscriber (in another terminal):

```
ros2 run turtle_controller position_subscriber
```

Run the smart controller:

```
ros2 run turtle_controller smart_controller
```

Optional teleop for interactive testing:

```
ros2 run teleop_twist_keyboard
teleop_twist_keyboard
```

C. Launch Files

Create a launch directory and launch file:

```
mkdir -p ~/ros2_ws/src/turtle_controller/launch
```

Create `turtle_lab.launch.py`:

```

from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='turtlesim',
            executable='turtlesim_node',
            name='turtlesim'
        ),
        Node(
            package='turtle_controller',
            executable='smart_controller',
            name='smart_controller'
        )
    ])

```

Update `setup.py` to include launch files:

```

import os
from glob import glob

```

```
data_files=[
    ('share/ament_index/resource_index/
packages', ['resource/' + package_name]),
    ('share/' + package_name, ['package.xml']),
    (os.path.join('share', package_name,
'launch'), glob('launch/*.launch.py')),
]
```

Run the launch file:

```
ros2 launch turtle_controller
turtle_ctrl.launch.py
```

IV. SUMMARY

Concept	Description
Package Structure	Organized collection of nodes, launch files, and dependencies
Node Inheritance	Extending rclpy.node.Node for ROS 2 functionality
Publishers	Nodes that send messages to topics
Subscribers	Nodes that receive messages from topics
Callbacks	Functions triggered by events (timers, messages)
Launch Files	Scripts to start multiple nodes simultaneously
Workspace	Development environment containing multiple packages
Colcon	Build system for ROS 2 packages

Command	Purpose
ros2 pkg create	Create new ROS 2 package
colcon build	Build ROS 2 packages
ros2 run <pkg> <executable>	Run a specific node
ros2 launch <pkg> <launch_file>	Run launch file
ros2 node list	List active nodes
ros2 node info <node>	Show node details
ros2 topic hz <topic>	Show topic frequency
ros2 topic bw <topic>	Show topic bandwidth